

AGH Intern Project Documentation :

Universal Retrieval-Augmented Generation (RAG) System using Groq API

1. Project Overview

This project implements a **Retrieval-Augmented Generation (RAG)** system capable of answering natural language queries from **user-provided documents** such as JSON, TXT, and PDF files. The system uses **semantic search** to retrieve relevant document content and a **large language model (LLM)** hosted on **Groq** to generate accurate, context-grounded responses.

Unlike traditional chatbots that rely on pre-trained knowledge, this RAG system ensures that **answers are generated strictly from uploaded documents**, reducing hallucination and improving trustworthiness.

2. Problem Statement

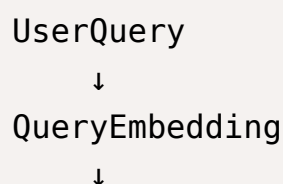
Large Language Models (LLMs) are powerful but suffer from:

- Hallucinations
- Lack of domain-specific knowledge
- Inability to reason over private documents

This project addresses these limitations by:

- Storing document knowledge in a **vector database**
 - Retrieving only relevant information
 - Passing retrieved context to the LLM at query time
-

3. System Architecture



```
FAISSVectorSearch
  ↓
Top-KRelevantChunks
  ↓
GroqLLM(LLaMA3.1)
  ↓
GroundedAnswer
```

4. Tools & Technologies Used (with justification)

4.1 Sentence Transformers (Embeddings)

Tool: `sentence-transformers/all-MiniLM-L6-v2`

Why used:

- Converts text into numerical vectors (embeddings)
- Captures semantic meaning, not just keywords
- Lightweight and fast
- Industry-standard for semantic search

Role in RAG:

- Used for embedding documents
- Used for embedding user queries

4.2 FAISS (Vector Database)

Tool: FAISS (Facebook AI Similarity Search)

Why FAISS:

- Extremely fast similarity search
- Optimized for large-scale vector retrieval
- Local, lightweight, no external service needed
- Supports Approximate Nearest Neighbor (ANN) search

Why NOT traditional databases:

- SQL databases cannot perform semantic similarity search

- Keyword matching is insufficient for natural language queries

Conclusion:

FAISS is ideal for **semantic document retrieval**, which is the backbone of RAG.

4.3 Groq API (LLM Inference)

Tool: Groq API

Model Used: `llama-3.1-8b-instant`

Why Groq:

- Extremely low-latency inference
- Free / generous developer tier
- Supports state-of-the-art LLaMA models
- Clean OpenAI-compatible API

Important Clarification:

- Groq is **not a model**
 - Groq is an **inference platform**
 - The actual intelligence comes from **LLaMA 3.1**
-

4.4 Python

Used as the orchestration language due to:

- Strong AI ecosystem
 - Excellent library support
 - Easy integration with FAISS and Groq
-

5. Project Structure

```
doc_RAG/  
|  
├─ core/  
|   ├── embedder.py  
|   ├── vector_db.py  
|   └── retriever.py
```

```
|   └─ groq_llm.py
|
|   └─ loaders/
|       ├── __init__.py
|       ├── json_loader.py
|       ├── text_loader.py
|       └─ pdf_loader.py
|
|   └─ data/
|       ├── colleges.json
|       ├── college_from_js.json
|       ├── sample.txt
|       └─ document.pdf
|
|   └─ build_index.py
|   └─ rag_chat.py
|   └─ requirements.txt
|   └─ .env
```

6. Phase-wise Implementation

Phase 1: Core RAG Pipeline

6.1 Document Loading (JSON Example)

```
def load_json(path):
    with open(path, "r", encoding="utf-8") as f:
        data = json.load(f)
    return [str(v) for v in data.values()]
```

6.2 Embedding Generation

```
def get_embedder():
    return HuggingFaceEmbeddings(
        model_name="sentence-transformers/all-MiniLM-L6-v2"
    )
```

6.3 Vector Database Creation

```
db = FAISS.from_texts(texts, embedder)
db.save_local("vector_store")
```

6.4 Query Retrieval

```
docs = db.similarity_search(query, k=3)
context = "\n".join([d.page_content for d in docs])
```

6.5 Groq LLM Integration

```
response = client.chat.completions.create(
    model="llama-3.1-8b-instant",
    messages=[{"role": "user", "content": prompt}],
)
```

Phase 2: Multi-Document Support

A universal loader was implemented to support multiple file formats.

```
def load_file(path):
    if path.endswith(".json"):
        return load_json(path)
    elif path.endswith(".txt"):
        return load_text(path)
    elif path.endswith(".pdf"):
        return load_pdf(path)
```

7. Crucial Errors Encountered & Solutions

Error 1: `ModuleNotFoundError: loaders`

Cause:

- Missing `__init__.py`

Fix:

```
loaders/__init__.py must exist
```

Error 2: FAISS Deserialization Error

```
ValueError: allow_dangerous_deserialization
```

Cause:

- FAISS uses pickle internally

Fix:

```
FAISS.load_local(path, embedder, allow_dangerous_deserializ  
ation=True)
```

Error 3: Groq API Key Not Found

```
GROQ_API_KEY environment variable not set
```

Cause:

- Environment variable not loaded

Fix:

```
from dotenv import load_dotenv  
load_dotenv()
```

Error 4: Groq Model Decommissioned

```
model llama3-8b-8192 has been decommissioned
```

Fix:

- Switched to:

```
llama-3.1-8b-instant
```

Error 5: JS Loader Path Errors (Windows + Node)

Problem:

- Node `require()` failed due to Windows path resolution
- Backslash escaping issues
- Temp directory execution

Engineering Decision:

- Convert JS → JSON once
- Avoid executing JS at runtime

Reasoning:

JavaScript files are executable code, not pure data.

Normalizing them into JSON improves safety, portability, and reliability.

8. Environment Configuration

`.env` file

```
GR0Q_API_KEY=gsk_XXXXXXXXXXXXXXXXXXXX
```

Accessed in code:

```
os.getenv("GR0Q_API_KEY")
```

9. Final Output

Example Query:

```
getme the locationof Ariyalur Engineering College
```

Example Response:

NH-227, Trichy-Chithambaram NH,
Karuppur-Senapathy Post,
Ariyalur District, Tamil Nadu